

ЛР 03. Модульне тестування

1. Мета

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %

2. Вихідний код реалізованого модуля з коментарями.

```
3. using System.Globalization;
4.
5. namespace LW3.Recommendation
6. {
7.     public class Restaurant
8.     {
9.         public string Name;
10.        public string CuisineType;
11.        public double Rating;
12.    }
13.
14.    public class RecommendationModule
15.    {
16.        // Обчислює рекомендаційний бал на основі рейтингу, відповідності
кухні,
17.        // цінового діапазону та кількості відгуків.
18.        public double CalculateRecommendationScore(double averageRating,
19.        bool likedCuisine, decimal price, int reviewCount)
20.        {
21.            double score = averageRating * 2;
22.
23.            // Якщо тип кухні відповідає уподобанням, додаємо додатковий
бал.
24.            if (likedCuisine)
25.            {
26.                score = score + 3;
27.            }
28.            else
29.            {
30.                score = score + 1;
31.            }
32.
33.            // Коригуємо бал залежно від ціни: дорогі ресторани трохи
знижуються,
34.            // дешеві отримують невеликий бонус.
```

```
34.     if (price > 20)
35.     {
36.         score = score - 1;
37.     }
38.     if (price < 10)
39.     {
40.         score = score + 1;
41.     }
42.
43.     // Якщо немає відгуків, зменшуємо надійність рекомендації.
44.     if (reviewCount == 0)
45.     {
46.         score = score - 2;
47.     }
48.
49.     return score;
50. }
51.
52. // Формує текстове подання інформації про ресторан.
53. public string GetRestaurantInfo(string name, string cuisineType, string
address, double rating, double averageReview, string mostExpensiveDish)
54. {
55.     return name + " (" + cuisineType + ") at " + address + ". Rating " +
rating.ToString(CultureInfo.InvariantCulture) + ". Average review " +
averageReview.ToString(CultureInfo.InvariantCulture) + ". Most expensive
dish " + mostExpensiveDish + ".";
56. }
57.
58. // Виконує пошук ресторанів за заданим типом кухні та мінімальним
рейтингом.
59. public List<Restaurant> SearchRestaurants(List<Restaurant>
knownRestaurants, string cuisineType, double minRating)
60. {
61.     List<Restaurant> result = new List<Restaurant>();
62.     if (knownRestaurants == null)
63.     {
64.         return result;
65.     }
66.
67.     for (int i = 0; i < knownRestaurants.Count; i++)
68.     {
69.         Restaurant restaurant = knownRestaurants[i];
70.         if (restaurant != null && restaurant.CuisineType == cuisineType &&
restaurant.Rating >= minRating)
71.         {
72.             result.Add(restaurant);
73.         }
```

```

74.    }
75.
76.    return result;
77.    }
78. }
79.}
80.

```

3. Таблиця проєктування тестів:

Метод	Параметр	Класи еквівалентності (EP)	Граничні значення (BVA)	Очікуваний результат
CalculateRecommendationScore	averageRating	EP1: нормальний рейтинг (2, 3, 4, 5)	BVA: 0, 5	score = averageRating * 2 + ...
CalculateRecommendationScore	likedCuisine	EP1: true EP2: false	BVA: немає	true додає 3, false додає 1
CalculateRecommendationScore	price	EP1: низька ціна EP2: середня ціна EP3: висока ціна	BVA: 10, 20	price < 10 → +1, price > 20 → -1
CalculateRecommendationScore	reviewCount	EP1: >0 EP2: 0	BVA: 0	0 → -2, >0 → без зниження
GetRestaurantInfo	name, cuisineType, address, mostExpensiveDish	EP1: нормальні непорожні рядки	BVA: порожній рядок	Правильна конкатенація рядка
GetRestaurantInfo	rating, averageReview	EP1: нормальні значення	BVA: 0, дробові значення	Форматування через InvariantCulture
SearchRestaurants	knownRestaurants	EP1: список ресторанів EP2: порожній список EP3: null	BVA: Count = 0, null	Null/порожній → порожній результат
SearchRestaurants	cuisineType	EP1: співпадає EP2: не співпадає	BVA: точний збір	Тільки співпадаючі кухні додаються
SearchRestaurants	minRating	EP1: Rating >= minRating EP2: Rating < minRating	BVA: Rating = minRating	Rating >= minRating → додається

4. Вихідний код тестового набору з коментарями.

```

using Xunit;
namespace LW3.Recommendation.Tests
{
    public class RecommendationModuleTests
    {
        // Техніка: EP (Equivalence Partitioning) + Positive
        // Опис: Перевірка базового розрахунку з звичайними параметрами
        [Fact]
        public void TestSimpleScore()
        {
            var module = new RecommendationModule();

```

```

    double result = module.CalculateRecommendationScore(3, true, 15, 5);
    Assert.Equal(9.0, result);
}

// Техніка: BVA (Boundary Value Analysis) + Negative
// Опис: Перевірка граничного випадку - 0 відзвітів (reviewCount == 0)
[Fact]
public void TestScoreWithNoReviews()
{
    var module = new RecommendationModule();
    double result = module.CalculateRecommendationScore(3, false, 12, 0);
    Assert.Equal(5.0, result);
}

// Техніка: EP (Equivalence Partitioning) + Positive
// Опис: Перевірка класу еквівалентності - улюблена кухня (likedCuisine =
true)
[Fact]
public void TestScoreLikedCuisine()
{
    var module = new RecommendationModule();
    double result = module.CalculateRecommendationScore(4, true, 15, 2);
    Assert.Equal(11.0, result);
}

// Техніка: EP (Equivalence Partitioning) + Positive
// Опис: Перевірка класу еквівалентності - нелюблена кухня (likedCuisine
= false)
[Fact]
public void TestScoreNotLiked()
{
    var module = new RecommendationModule();
    double result = module.CalculateRecommendationScore(2, false, 15, 3);
    Assert.Equal(5.0, result);
}

// Техніка: EP (Equivalence Partitioning) + Positive
// Опис: Перевірка середньої оцінки в нормальному діапазоні
[Fact]
public void TestScoreMiddleRating()
{
    var module = new RecommendationModule();
    double result = module.CalculateRecommendationScore(2, true, 15, 4);
    Assert.Equal(7.0, result);
}

// Техніка: Positive

```

```

// Опис: Перевірка форматування інформації про ресторан
[Fact]
public void TestGetRestaurantInfo()
{
    var module = new RecommendationModule();
    string result = module.GetRestaurantInfo("Pizza Place", "Italian", "Main St",
4.5, 4.2, "Margherita");
    Assert.Contains("Pizza Place", result);
}

// Техніка: Positive
// Опис: Перевірка форматування з іншими параметрами
[Fact]
public void TestGetRestaurantInfoCuisine()
{
    var module = new RecommendationModule();
    string result = module.GetRestaurantInfo("Sushi Bar", "Japanese", "Oak
Ave", 4.8, 4.6, "Salmon Roll");
    Assert.Contains("Japanese", result);
}

// Техніка: BVA (Boundary Value Analysis) + Negative
// Опис: Перевірка граничного випадку - null список (knownRestaurants ==
null)
[Fact]
public void TestSearchRestaurantsWithNullList()
{
    var module = new RecommendationModule();
    List<Restaurant> result = module.SearchRestaurants(null, "Italian", 3.0);
    Assert.Empty(result);
}

// Техніка: BVA (Boundary Value Analysis) + Negative
// Опис: Перевірка граничного випадку - порожній список
[Fact]
public void TestSearchRestaurantsEmpty()
{
    var module = new RecommendationModule();
    var list = new List<Restaurant>();
    List<Restaurant> result = module.SearchRestaurants(list, "Italian", 3.0);
    Assert.Empty(result);
}

// Техніка: EP (Equivalence Partitioning) + Positive
// Опис: Перевірка високої оцінки
[Fact]
public void TestScoreWithHighRating()

```

```

{
    var module = new RecommendationModule();
    double result = module.CalculateRecommendationScore(5, true, 15, 10);
    Assert.Equal(13.0, result);
}
}
}

```

5. Покриття коду з відображенням відсотку line coverage.

The screenshot displays the Visual Studio Test Explorer and the Code Coverage window. The Test Explorer shows a successful test run for 'Lab3' with 10 tests passing. The Code Coverage window shows the following results:

Иерархия	Охват (% блоков)	Охват (% строк)
ASU_DESKTOP-0N70AEL_2026-06-03.15_13_49.coverage	86,2%	84,8%

6. Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/ruslankhyzhniakov-lang/LW3>

7. Висновки

У проєкті 10 юніт-тестів, і всі вони успішно пройшли.

Покриття коду наразі таке:

- блоків – 86.2%
- рядків – 84.8%

Основна логіка модуля працює і частково покрита тестами: обчислення рекомендаційного балу, пошук ресторану (включаючи null та порожній список), а також форматування інформації про ресторан.

Виявлені проблеми

CalculateRecommendationScore недостатньо перевіряє граничні випадки (price = 10 і 20, негативні або нульові averageRating, reviewCount < 0).

SearchRestaurants майже не тестує:

- null-елементи всередині списку ресторанів,
- пошук по несумісному типу кухні,
- нечутливість до регістру.
- Відсутня валідація вхідних параметрів (null для name, address, mostExpensiveDish тощо).

Що можна покращити

\

- Розширити тести
- Тестувати список з одним null рестораном.
- Перевірити пошук по іншому типу кухні.
- Замінити публічні поля в Restaurant на нормальні властивості (public string Name { get; set; }).
- Поліпшити покриття
- Покрити всі гілки умов у CalculateRecommendationScore.
- Додати тести на різні сценарії форматування.